



cpi'fp

Los Enlaces

DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

INTRODUCCIÓN A LARAVEL

1. FRAMEWORKS

Un **framework** es un conjunto estandarizado de prácticas de programación para resolver una serie de problemas habituales.

El framework proporciona una serie de **clases, librerías y otros componentes** para facilitar el desarrollo ágil, seguro y escalable de nuevas aplicaciones.

Por lo tanto, un **framework MVC PHP** es un conjunto de clases, librerías y otros componentes destinados a servir de base para desarrollar aplicaciones en PHP con arquitectura MVC.

1. FRAMEWORKS

Ventajas:

- Reutilización del trabajo ya hecho.
- Extensa documentación.
- Separación en capas.
- Seguimiento de buenas prácticas de programación.
- Escalabilidad y mantenimiento.
- Desarrollo más rápido y, por tanto, más económico.

1. FRAMEWORKS

Inconvenientes:

- A veces pueden limitar el desarrollo.
- Curva de aprendizaje costosa (más en unos que en otros).
- Puede llegar a implicar más trabajo, dependiendo del proyecto.
- Preferencias personales: algunos programadores solo se sienten cómodos si todo el código es suyo.
- Actualizaciones frecuentes e imprevistas.
- Ocultan gran parte del funcionamiento de la aplicación: no son aptos para aprender a programar.

1. FRAMEWORKS

En resumen:

- La idea es que, al usar un framework, solo te centras en desarrollar lo importante. El resto (lo que ya estaba desarrollado en el framework y que es común a muchas aplicaciones web) no te quita tiempo.

Algunos frameworks para PHP:

- **Symfony**: el más extendido desde hace años.
- **Laravel**: el que tiene un crecimiento más rápido.
- **CodeIgniter**: el más sencillo (su implantación en industria es menor).

2. CARACTERÍSTICAS DE LARAVEL

Laravel es un framework PHP MVC para desarrollo rápido de aplicaciones web. Automatiza muchos procesos habituales y tiene una curva de aprendizaje pronunciada, pero no tanto como otros frameworks (con “otros” queremos decir “Symfony”, que es, sin duda, el framework más difícil de aprender).

Desde hace algunos años, Laravel ha experimentado un crecimiento espectacular en el mercado de las aplicaciones web.

2. CARACTERÍSTICAS DE LARAVEL

Ventajas:

- Sintaxis simple y elegante.
- Mapeo objeto-relacional (ORM): Eloquent.
- Potente sistema de plantillas para vistas: Blade.
- Reutiliza y moderniza componentes de Symfony.
- Es sencillo (esto es un decir) y potente.
- Es previsible que domine el mercado durante los próximos años.
- Comunidad de usuarios altamente especializada.

2. CARACTERÍSTICAS DE LARAVEL

Inconvenientes:

- Instalación, configuración y despliegue complejos.
- Curva de aprendizaje elevada.
- Se mueve según los intereses personales de su autor, con actualizaciones muy frecuentes y cambios caprichosos.
- Inestabilidad de varios de sus componentes: a menudo hay que recurrir a *fixes* o a componentes de terceros.
- Fuerte dependencia de la consola de comandos y de herramientas de terceros (composer, vagrant, npm...).

3. INSTALACIÓN

Requisitos:

- Versión PHP 8.1. o superior (8.2. con XAMPP)
- Sistema gestor de BBDD (MySQL incluido en XAMPP)
- Composer (gestor de paquetes de PHP)
- Node.js (entorno de ejecución para servidor en JS). Laravel utiliza un compilador JS (Vite, palabra en francés para "rápido").
- Editor de código (en nuestro caso, VSCode)

3. INSTALACIÓN

Instalación de Composer:

- <https://getcomposer.org/>

Windows Installer

The installer - which requires that you have PHP already installed - will variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest comp



A Dependency Manager for PHP

Latest **2.6.6** ([changelog](#))

[Getting Started](#)[Download](#)[Documentation](#)[Browse Packages](#)[Issues](#)[GitHub](#)

Authors: [Nils Adermann](#), [Jordi Boggiano](#) and many [community contributions](#)

Sponsored by:



PRIVATE PACKAGEIST

Logo by: [WizardCat](#)

3. INSTALACIÓN

Instalación de Composer:

- En la pantalla *Settings Check* hay que indicar la ruta al archivo *php.exe*, situado en el directorio `.../xampp/php`, y marcar el checkbox *Add this PHP to your path?*
- En el resto de pantallas solo hay que darle a *Next* y en la última a *Finish*.
- En la consola podemos comprobar que se ha instalado ejecutando el comando `composer` desde cualquier ruta.

3. INSTALACIÓN

Instalación de Node.js:

- <https://nodejs.org/en>

Node.js® is an open-source, cross-platform JavaScript runtime environment.

Download Node.js®

20.11.0 LTS Recommended For Most Users	21.6.0 Current Latest Features
--	--

[Other Downloads](#) | [Changelog](#) | [API Docs](#) [Other Downloads](#) | [Changelog](#) | [API Docs](#)

For information about supported releases, see the [release schedule](#).

3. INSTALACIÓN

Instalación de Node.js:

- En la pantalla *Destination Folder*, acordaos de cambiar la ruta a vuestra carpeta personal en la partición D:/
- En la pantalla *Tools for Native Modules* hay que marcar el checkbox *Automatically install the necessary tools*.
- En el resto de pantallas *Next* y *Finish*. Se abrirá la consola de comandos y simplemente pulsamos *Intro* para continuar.

3. INSTALACIÓN

Creación de un proyecto en Laravel:

- <https://laravel.com/docs/10.x>

Creating a Laravel Project

Before creating your first Laravel project, make sure that your local machine has PHP and [Composer](#) installed. If you are developing on macOS, PHP and Composer can be installed in minutes via [Laravel Herd](#). In addition, we recommend [installing Node and NPM](#).

After you have installed PHP and Composer, you may create a new Laravel project via Composer's `create-project` command:

```
composer create-project laravel/laravel example-app
```

3. INSTALACIÓN

Creación de un proyecto en Laravel:

- Para crear un nuevo proyecto de Laravel debemos situarnos en el directorio de nuestro servidor local (Apache).
- Vamos a abrir una terminal de Git BASH desde .../htdocs/laravel y ejecutamos el siguiente comando:

- Abrimos en el navegador <http://localhost/laravel/example-app/>

3. INSTALACIÓN

Creación de un proyecto en Laravel:

- Podemos ver entonces el esqueleto de directorios del proyecto.



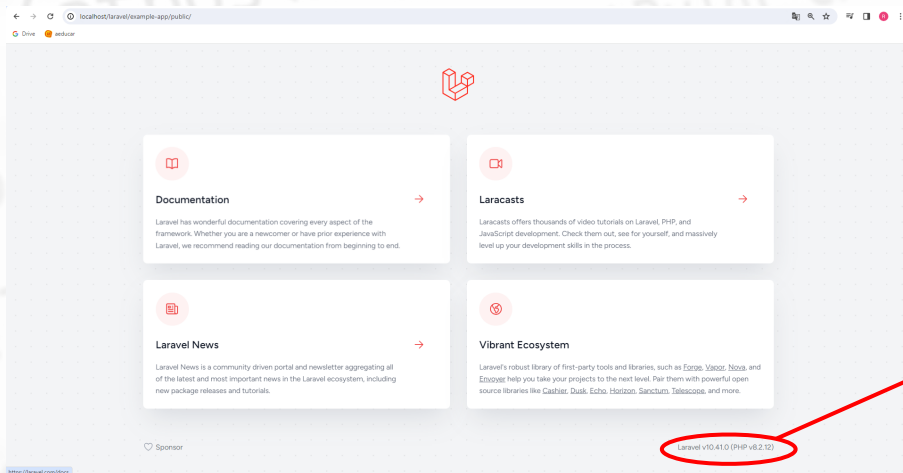
The screenshot shows a web browser window with the address bar displaying 'localhost/laravel/example-app/'. Below the address bar, there is a navigation bar with 'Drive' and 'eduard' buttons. The main content area is titled 'Index of /laravel/example-app' and contains a table listing the files and directories in the project.

Name	Last modified	Size	Description
 Parent Directory		-	
 app/	2024-01-04 17:47	-	
 artisan	2024-01-04 17:47	1.6K	
 bootstrap/	2024-01-04 17:47	-	
 composer.json	2024-01-04 17:47	1.8K	
 composer.lock	2024-01-17 15:58	292K	
 config/	2024-01-04 17:47	-	
 database/	2024-01-04 17:47	-	
 package.json	2024-01-04 17:47	248	
 phpunit.xml	2024-01-04 17:47	1.1K	
 public/	2024-01-04 17:47	-	
 resources/	2024-01-04 17:47	-	

3. INSTALACIÓN

Creación de un proyecto en Laravel:

- En la carpeta /public se muestra el contenido de la aplicación.



**Versiones de Laravel
y PHP instaladas**

3. INSTALACIÓN

Creación de un proyecto en Laravel:

- Otra forma de crear nuevos proyectos en Laravel es incluyendo el instalador global de Laravel con el siguiente comando (éste se puede ejecutar desde cualquier directorio):
- Posteriormente, solo tendremos que ejecutar:

3. INSTALACIÓN

Creación de un proyecto en Laravel:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\ruben\xampp\htdocs\laravel> laravel new blog

  Laravel

Would you like to install a starter kit? [No starter kit]:
[none] No starter kit
[breeze] Laravel Breeze
[jetstream] Laravel Jetstream
>

Which testing framework do you prefer? [PHPUnit]:
[0] PHPUnit
[1] Pest
>

Would you like to initialize a Git repository? (yes/no) [no]:
>

Creating a "laravel/laravel" project at "./blog"
Installing laravel/laravel (v10.3.2)
```

3. INSTALACIÓN

Extensiones de VSCode para Laravel:

- Laravel Blade formatter
- Laravel Blade Snippets
- Laravel Snippets
- Laravel goto view
- PHP Intelephense

3. INSTALACIÓN

Laravel Blade formatter:

- Nos permite formatear el código. Especialmente útil cuando copiamos código de otras fuentes.



Laravel Blade formatter v0.24.0

Shuheï Hayashibara  [shufo.dev](#)

 1,178,603

★★★★★ (26)

♥ Sponsor

Laravel Blade formatter for VSCode

Install



3. INSTALACIÓN

Laravel Blade Snippets:

- Proporciona snippets (fragmentos de código) mientras programamos para agilizar el proceso de escribir comandos en Blade.



Laravel Blade Snippets

v1.34.0

Winnie Lin

3,115,453

★★★★☆ (37)

Laravel blade snippets and syntax highlight support

Install



3. INSTALACIÓN

Laravel Snippets:

- Proporciona snippets para Laravel (sugerencias de código para rutas, controladores, vistas,...).



Laravel Snippets

v1.17.0

Winnie Lin

🔗 2,054,233

★★★★★ (11)

Laravel snippets for Visual Studio Code (Support Laravel 5 and above)

Install



3. INSTALACIÓN

Laravel goto view:

- Implementa una funcionalidad de navegación por las vistas pulsando CTRL y clickando sobre las partes de código que las invoquen.



Laravel goto view

v1.3.11

codingyu



1,500,592

★★★★☆ (21)

Quick jump to view

Install



3. INSTALACIÓN

PHP Intelephense:

- Incluye diversas ayudas al escribir código PHP, como importación automática de clases, controladores,...



PHP Intelephense v1.10.2

Ben Mewburn  intelephense.com

 10,729,977

★★★★★ (368)

 Sponsor

PHP code intelligence for Visual Studio Code

Install



4. ARQUITECTURA Y CONFIGURACIÓN

Estructura de directorios:

- **composer.json**: información para composer (administrador de paquetes de PHP). Sirve para instalar todas las librerías de terceros que Laravel necesita para funcionar.
- **/app**: código de nuestra aplicación (aquí están los modelos).
- **/app/config**: configuración de la aplicación.
- **/app/http**: peticiones HTTP, incluyendo los controladores.
- **/plugins**: <https://elfsight.com/es/laravel-modules/>

4. ARQUITECTURA Y CONFIGURACIÓN

Estructura de directorios:

- **/database:** migraciones y *seeders* de la base de datos.
- **/public:** directorio de acceso público. Aquí Laravel generará todo lo que hay que mover al servidor para poner la web en producción. Puedes crear aquí dentro carpetas para colocar imágenes, scripts JS o archivos css.
- **/storage:** aquí Laravel guarda su memoria caché, información sobre las sesiones, vistas compiladas... **No debes tocar esta carpeta.**

4. ARQUITECTURA Y CONFIGURACIÓN

Estructura de directorios:

- **/resources:** Aquí dentro están las vistas. También el resto de *assets* (imágenes, css, js). A diferencia del directorio `/public`, los archivos JS o css que coloquemos aquí estarán precompilados, no serán accesibles vía web y Laravel se encargará de compilarlos automáticamente y generar versiones minimizadas de nuestro CSS y nuestro JS. De momento, vamos a colocar nuestro CSS y JS en la carpeta `/public`. Más adelante puedes trastear con `/resources` si lo deseas.

4. ARQUITECTURA Y CONFIGURACIÓN

Estructura de directorios:

- **/vendors:** librerías de terceros. Es importante añadir esta carpeta al `.gitignore` si vas a construir un repositorio git para tu aplicación Laravel, porque `/vendors` puede ocupar bastante espacio y no tiene sentido incluirla en tu proyecto. Si necesitas desplegar esta aplicación Laravel en otro servidor, basta con clonar el repositorio y ejecutar `composer update`. Eso rellenará la carpeta `/vendor` con las librerías más adecuadas para ese servidor.

4. ARQUITECTURA Y CONFIGURACIÓN

Convenciones en Laravel:

- **Identificadores:** en inglés, mejor *User* que *Usuario*.
- **Modelos:** igual a los de las tablas de la base de datos pero en singular, en CamelCase y con mayúscula al principio.
Ejemplo: *RegisteredUser*
- **Controladores:** como los modelos, pero añadiendo la palabra “controller”.
Ejemplo: *RegisteredUserController*

4. ARQUITECTURA Y CONFIGURACIÓN

Convenciones en Laravel:

- **Métodos:** se nombran en camelCase y con minúscula al principio.
Ejemplo: *RegisteredUser::getAll()*
- **Atributos:** se nombran en snake_case y con minúscula al principio.
Ejemplo: *RegisteredUser::first_name*
- **Variables:** los identificadores deben ir en camelCase y empezando con minúscula. En plural si se trata de una colección y en singular si es un objeto individual o una variable simple.
Ejemplo: *bannedUsers* (colección), *articleContent* (objeto individual)

4. ARQUITECTURA Y CONFIGURACIÓN

Convenciones en Laravel:

- **Tablas:** se nombran en snake_case y en plural.
Ejemplo: *registered_users*
- **Columnas de las tablas:** se nombran en snake_case, sin referencia al nombre de la tabla.
Ejemplo: *first_name*
 - **Clave primaria:** la llamaremos siempre *id*, de tipo integer y auto-increment.

4. ARQUITECTURA Y CONFIGURACIÓN

Convenciones en Laravel:

- **Columnas de las tablas:**
 - **Claves ajenas:** Se forman con el nombre de la tabla ajena en singular más la palabra *id*.
Ejemplo: *article_id*
 - **Timestamps:** Laravel siempre crea marcas de tiempo para todo. Y siempre se llaman *created_at* y *updated_at*, de tipo *datetime*. Acostúmbrate a tenerlas en todas tus tablas.

4. ARQUITECTURA Y CONFIGURACIÓN

Variables de entorno (archivo `.env`):

- **APP_ENV**: en esta variable se indica si la aplicación está en desarrollo o en producción.
- **APP_DEBUG**: muestra los errores para depuración. Se pone a *true* durante el desarrollo y se cambia a *false* al pasar a producción.
- **APP_URL**: la URL base de la aplicación.
- **DB_CONNECTION, DB_HOST, DB_USERNAME, etc**: configuración de la conexión a la base de datos.

4. ARQUITECTURA Y CONFIGURACIÓN

Variables de entorno (archivo `.env`):

- Una instalación limpia de Laravel vendrá con un archivo llamado `.env.example`, que contiene una plantilla para que puedas construir tu propio archivo `.env`. También viene con un archivo `.env` que será el que tome por defecto para la configuración del entorno del proyecto.
- Cuidado: el archivo `.env` NO debe sincronizarse con **git** (o con el control de versiones que usemos) porque contiene información sensible. Asegúrate de incluirlo en tu `.gitignore`.

4. ARQUITECTURA Y CONFIGURACIÓN

Variables de entorno (archivo `.env`):

- Una vez creado nuestro archivo `.env`, podemos usar las variables definidas en él en cualquier otra parte de la aplicación. Por ejemplo, en `/config/database.php` usaremos una expresión así:

```
'default' => env('DB_CONNECTION', 'mysql')
```

- El primer parámetro de `env()` es la variable de entorno que queremos consultar y el segundo es el valor por defecto en caso de que la variable no exista.

4. ARQUITECTURA Y CONFIGURACIÓN

Archivos de configuración (/config):

- **database.php**: configuración de la conexión a la base de datos. Toma sus valores principales de .env, pero desde aquí podemos cambiar otras cosas, como el controlador (por defecto es MySQL, pero podemos cambiarlo a PostgreSQL, SQLite, etc).
- **app.php**: nombre de la aplicación, estado (desarrollo, producción...).
- **session.php**: forma en la que se almacenarán las variables de sesión (por defecto, en un fichero en el servidor).